

Documentación del Trabajo Realizado en la Pasantía

Introducción

En el marco de la pasantía, el objetivo principal fue estudiar, implementar y validar el uso de ontologías en combinación con herramientas de la web semántica y técnicas de Machine Learning. Este documento tiene como objetivo registrar todo el proceso llevado a cabo, incluyendo el aprendizaje teórico, la revisión de literatura relevante, y la implementación práctica mediante la herramienta CatE aplicada a una ontología personalizada.

1. Revisión Teórica

1.1. Conceptos Básicos de Ontologías

Definición y propósito de las ontologías: Las ontologías son representaciones formales y explícitas de un conjunto de conceptos dentro de un dominio y de las relaciones que existen entre ellos. Estas permiten estructurar el conocimiento de manera organizada y facilitar su procesamiento por sistemas computacionales.

- Una ontología proporciona un lenguaje estándar y formal para describir dominios complejos, facilitando la interoperabilidad y el razonamiento automático.
- La descripción lógica (DL) permite expresar semántica de conceptos con precisión, brindando las bases para operaciones de inferencia y validación.

Rol en el contexto de la Web Semántica: En el marco de la Web Semántica, las ontologías permiten organizar y vincular datos estructurados de manera semántica, facilitando su interpretación automática por agentes inteligentes. Tecnologías como RDF (Resource Description Framework), OWL (Web Ontology Language) y SPARQL se utilizan para representar y consultar ontologías.

- **Principales lenguajes de representación:**
 - **RDF (Resource Description Framework):** Lenguaje base para la representación de información y metadatos en grafos.
 - **OWL (Web Ontology Language):** Extensión de RDF que permite definir clases, propiedades, axiomas y restricciones formales en un dominio.
 - **SPARQL (SPARQL Protocol and RDF Query Language):** Lenguaje de consultas utilizado para recuperar y manipular información en RDF.

1.2. Ontologías y Machine Learning

- **Beneficios de combinar ontologías con ML:** La integración de ontologías con Machine Learning (ML) aporta beneficios significativos, como la capacidad de agregar contexto semántico a los datos, mejorando la interpretabilidad y precisión de los modelos. Las ontologías permiten:
 - **Mejorar la calidad de los datos:** Al proporcionar una representación estructurada y semántica del dominio, los datos utilizados en los modelos ML se vuelven más coherentes y relevantes.
 - **Interpretabilidad:** Las ontologías aportan un marco conceptual que facilita la interpretación de los resultados generados por los modelos de ML.
 - **Reducción de datos redundantes:** Mediante la inferencia automática, se pueden generar nuevas relaciones a partir de la información existente.
 - **Facilitación del razonamiento explícito:** Al combinar descripciones lógicas con modelos predictivos, es posible automatizar procesos de inferencia y razonamiento para encontrar patrones ocultos.
- **Ejemplos de casos de uso en sistemas inteligentes:**
 - **Sistemas de recomendación:** Utilizan ontologías para agregar contexto semántico a los perfiles de usuario, mejorando las recomendaciones generadas por los algoritmos de ML.
 - **Procesamiento de lenguaje natural (NLP):** Las ontologías ayudan a estructurar el conocimiento semántico en tareas como la clasificación de textos y el reconocimiento de entidades.
 - **Sistemas de diagnóstico médico:** Combinan ontologías médicas (como SNOMED CT) con modelos ML para detectar patrones y facilitar diagnósticos precisos.
 - **Robótica:** Los robots utilizan ontologías para entender y razonar sobre el entorno, permitiendo la toma de decisiones más informadas.

1.3. Herramientas y Tecnologías Relacionadas

- **CatE:**

CatE (Categorical Embeddings) es una herramienta diseñada para generar embeddings de ontologías basadas en lógicas descriptivas (DL) y representarlas en un espacio vectorial. Este enfoque permite utilizar técnicas de machine learning en grafos generados a partir de ontologías, facilitando tareas como la predicción de axiomas y la inferencia semántica.

- **Principales funcionalidades:**

- Permite transformar axiomas ontológicos en proyecciones de grafos y, posteriormente, aplicar métodos de graph embeddings.
- Facilita el uso de embeddings para tareas de razonamiento aproximado y predicción de relaciones en ontologías.
- Soporta la integración con bibliotecas como mOWL para manejar ontologías OWL y realizar procesos de embedding eficientes.

- **mOWL:**

La biblioteca mOWL (machine learning for OWL ontologies) proporciona herramientas para aplicar métodos de ML en ontologías formateadas en OWL. Incluye implementaciones de modelos como OWL2Vec* y facilita la proyección de axiomas y su posterior embedding.

- **Principales características:**

- Generación de grafos ontológicos a partir de axiomas OWL.
- Integración con algoritmos de knowledge graph embeddings como TransE, TransR, y otros.
- Proporciona métodos para evaluar y validar los embeddings generados.

- **Integración con CatE:**

mOWL fue una herramienta clave para trabajar con las ontologías, ya que permitió preprocesar la ontología y generar las proyecciones de grafos requeridas por CatE.

- **Anaconda:**

Es una distribución de Python y R que incluye herramientas para el desarrollo y la gestión de entornos en ciencia de datos, machine learning, y análisis.

Se utilizó para la gestión de entornos virtuales y bibliotecas durante el desarrollo. Esto permitió aislar las dependencias de CatE, mOWL y otras herramientas del sistema operativo principal.

- **OWL2Vec***

Es una técnica basada en **embedding** que utiliza modelos de aprendizaje profundo para representar ontologías expresadas en OWL (Web Ontology Language) en un espacio vectorial. Combina ideas de procesamiento de lenguaje natural (NLP) y aprendizaje de representaciones para convertir conceptos y relaciones de una ontología en vectores densos, lo que facilita su uso en tareas de machine learning y análisis semántico.

Se utilizo para generar los archivos de testeo, validación, `classes.txt`, `individuals.txt` (Clases e individuos de la ontología) e `inferred_classes.txt` (Clases inferidas para cada individuo)

2. Seteo del entorno

Como aclaración, yo trabaje en Windows por lo que todos los comandos, así como capturas son sobre este sistema.

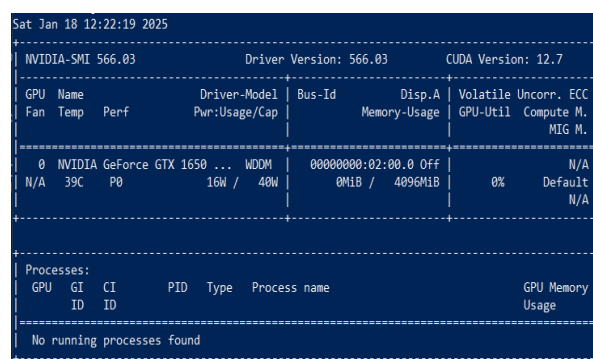
Lo primero para tener en cuenta es que el entrenamiento de la red neuronal precisa de una computadora con soporte CUDA. En mi caso, la tarjeta de video que tenía mi laptop no lo tenía por lo cual tuve que cambiar a otra que tenía una nvidia geforce gtx.

Nota: Según información que encontré en internet, así como en la página de pyTorch existe la alternativa de realizar el entrenamiento haciendo uso de la CPU en vez de la GPU. Yo lo intente y no me funcionó por lo que no recomendaría usarlo a menos que sea la única opción que se tiene ya que más allá de que pueda funcionar o no, el entrenamiento será mucho más lento.

catE hace uso de pytorch una librería muy utilizada en machin learning. Para instalarlo se debe ir a la siguiente página <https://pytorch.org/get-started/locally/> y seleccionar la opción compatible con nuestra versión CUDA. Para ver que versión tenemos se debe abrir la powershell y poner el siguiente comando:

```
nvidia-smi
```

Lo cual debe desplegar algo como esto



```
Sat Jan 18 12:22:19 2025
```

NVIDIA-SMI 566.03		Driver Version: 566.03		CUDA Version: 12.7	
GPU	Name	Driver-Model	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util Compute M.
				MIG M.	
0	NVIDIA GeForce GTX 1650	WDDM	00000000:02:00:0	Off	N/A
N/A	39C	P0	16W / 40W	0MiB / 4096MiB	0% Default
				N/A	

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID					
No running processes found						

En el caso de la computadora que utilicé, la versión de CUDA es 12.7 pero como no hay disponible un pyTorch específico para ella use la que era más cercana a la mia(ingrese `pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124` en la powershell). Todas las instalaciones pueden hacerse también desde la powershell de anaconda. Ocurre que en ocasiones el pip da error a la hora de instalar cosas por lo que sirve usar conda.

De todas maneras, sin tener soporte CUDA es posible realizar todo el resto del proceso (la generación de los archivos necesarios para el entrenamiento).

Es necesario descargar owlapi

<https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution>, hay que agregar la dirección de donde se instaló. También se debe descargar SLF4J, cambiarlo a la ruta donde está el owlapi y poner su path en el archivo `run_cat_completion.py`. SLF4J no se descarga como un archivo ejecutable o un instalador. En su lugar, se incluye como una dependencia en tu proyecto de desarrollo. Esto se hace agregándolo a un `pom.xml`.

<https://repo1.maven.org/maven2/org/slf4j/>

También se debe de tener instalado python (versión 3.9 o superior), JDK (versión 22 o superior). En mi caso tuve muchos problemas con java por lo que no solo instalé JDK sino que también open JDK (version = 8.0.152) y posteriormente a esto se debi crear una nueva variable de entorno con el nombre `JAVA_HOME` y la dirección donde se encuentra instalado java en el sistema, en mi caso se encontraba en `C:\Program Files\Java\jre1.8.0_361` y luego agregar al path `%JAVA_HOME%\bin`. Como dije anteriormente, yo particularmente tuve problemas con java por lo que por más de que `JAVA` fuera una variable del entorno, en el archivo `run_cat_completion.py` tuve que indicar específicamente donde se encontraba el archivo `jvm.dll`

Originalmente el archivo a partir de la línea 6 se veía así:

```
import mowl
mowl.init_jvm("10g")
import click as ck
```

Luego tuve que agregarle en la inicialización de `jvm` la dirección donde se encuentra mi archivo `jvm.dll` por lo que quedo así:

```
import mowl
mowl.init_jvm("10g")
por mowl.init_jvm("C:\Program Files\Java\jdk-23\bin\server\jvm.dll", "-ea", "-Xmx10g")
import click as ck
```

Nota: el archivo `jvm.dll` siempre se encuentra dentro de la carpeta ruta\hasta\carpetaJava\Java\jdk-version\bin\server\jvm.dll

A su vez, antes de comenzar hay que asegurarnos de tener Windows build Path instalado. De no tenerlo, va a saltar un mensaje de error como el siguiente:

```
error: Microsoft Visual C++ 14.0 or greater is required. Get it with
"Microsoft C++ Build Tools":
https://visualstudio.microsoft.com/visual-cpp-build-tools/
```

```
[end of output]
```

Lo que se debe hacer es entrar en <https://visualstudio.microsoft.com/es/visual-cpp-build-tools/> e instalar microsoft c++ build tools.

Las últimas dos herramientas por instalar antes de empezar a armar el entorno (e instalar en las correspondientes librerías) son anaconda y eclipse ide. Anaconda es donde desarrollaremos los entornos mowl, catE y OWL2Vec*. Por otro lado, necesitaremos eclipse para que con OWL2Vec* consigamos algunos de los archivos a usar.

Ahora que ya tengamos todo lo anterior instalado empezaremos por crear lo entornos. Dentro de anaconda debemos abrir el powershell y para crear el entorno de

-mOWL:

```
pip install git+https://github.com/bio-ontology-research-group/mowl
```

con este comando obtendremos el repositorio de mowl en nuestra computadora.

```
conda create -n mowl_env
```

de esta manera crearemos el entorno dentro de anaconda. Cada vez que queramos activarlo es necesario escribir **conda activate mowl_env** y para desactivarlo **conda deactivate mowl_env**.

```
pip install torch==4.3.0 JPype1==1.5.1 pykeen==1.11.0 scipy<1.15.0
```

la primera vez que se activa mowl_env debemos usar el comando anterior para instalar las dependencias necesarias para usar mowl.

-catE:

Primero se debe clonar el repositorio de catE y luego navegar hasta donde se encuentre guardado.

```
cd catE
```

nos movemos al directorio donde localizamos catE

```
conda env create -f environment.yml
```

creamos el entorno

conda activate/deactivate cate : para activar/desactivar el entorno

-OWL2vec:

```
git clone https://github.com/KRR-Oxford/OWL2Vec-Star.git
```

Para clonar owl2vec en nuestra computadora

```
cd ruta\a\la\carpeta\OWL2Vec-Star
```

Para colocarnos en la carpeta donde se encuentra owl2Vec

```
conda create -n owl2vec python=3.8
```

De esta forma creamos el entorno en Anaconda

```
conda activate owl2vec
```

Activamos el entorno

```
pip install -r requirements_owl2vec.txt
```

Instalamos todos los requerimientos del entorno

Si por alguna razón no se instalan todos los paquetes que se requieren puede hacerse manualmente uno por uno. En mi caso tuve un problema con numPy por lo que a partir de la línea 6 no se me instalaron los paquetes y tuve que irlo haciendo de a uno.

El error que tuve fue por la versión de NumPy instalada (1.26.4) no incluía el módulo `numpy.distutils`, que es requerido para instalar scikit-learn versión 0.24.2. La solución fue instalar una versión anterior de NumPy que incluya ese módulo. Por lo que tuve que desinstalar numPy y obligar a que se instale una versión anterior (instalé la 1.19.5)

Luego ejecuté

```
python setup.py install para instalarlo
```

Modifiqué el archivo `default_multi.cfg` para especificar la ruta a la ontología así como donde quería que se guardara la tripleta generada.

```
[BASIC]
# the file of input ontology; mandatory; you can also set it as a projected
ontology
ontology_file = D:\mai\catE\use_cases\ontologia\data\ontologia.owl
```

Lo último que vamos a instalar es dentro del entorno de catE (por lo tanto, debemos activar el entorno y movernos a donde se ubica catE).

Pondremos en la powerShell:

```
pip install wandb -upgrade
```

con esto vamos a conseguir instalar wandb que es una plataforma para rastrear, visualizar y comparar experimentos de aprendizaje automático. Cuando la usemos debemos de tener un usuario por lo que se debe buscar wandb en google y registrarse.

3. Recorrido hecho

A lo largo de estas semanas di muchas vueltas intentando descifrar como conseguir los diferentes archivos que se necesitan para usar catE. En un principio intente ver si podía crear por mí misma los archivos de validación, testeo y entrenamiento, pero me pareció que la forma debía de ser más automática. Luego de leer los archivos dentro del repositorio y no encontrar nada decidí enviar un [email](#) a los que realizaron el paper de catE para consultarles si había algo que no está teniendo en cuenta, a lo que respondieron que faltaba el script responsable de hacerlo (por eso no había conseguido encontrar como generarlos) y lo subieron al repositorio (el archivo se encuentra en la carta src y su nombre es `create_subsumption_dataset.py`). Una vez subieron este archivo conseguí obtener el archivo `train.csv` pero seguía faltándome los archivos de validación y testeo. Seguí buscando e intentando entender que me faltaba. Conseguí crear el archivo de clases, pero nada más. Después de unos días de dar vueltas sin llegar a la solución decidí mandarles otro email consultando a lo que nuevamente me dijeron que faltaba y averiguando más sobre el funcionamiento a lo que me respondieron que faltaba un archivo, así como también que ellos habían usado OWL2vec* para generar los archivos. Luego de esto, estuve viendo cómo funcionaba owl2vec* y al principio solo conseguí los archivos `annotations/axioms/entities/projection`. Buscando dentro del repositorio vi que había una carpeta `use_studies` donde había un readme que decía que ellos habían generado sus archivos de `train/test/validacion` así como los anteriores que ya había conseguido usando el archivo `java ontology_embed.ClassAssertion_NormalSplit`. Para usarlo, dentro de la carpeta `java` donde está ubicado el archivo hay un readme que sugiere construir un proyecto de maven con ciertas dependencias para poder correrlo bien. A continuación, explico cómo hacer:

Yo tenía en mi computadora instalado eclipse, por lo cual el proyecto lo cree directamente en este. Para eso hay que ir a `file>new>maven project`. Luego seleccioné que fuera un proyecto simple, un nombre y le di a finalizar. Luego se debe abrir el proyecto,

Esta dependencia me dio problemas

```
<dependency>
<groupId>ontology-services-toolkit</groupId>
<artifactId>ontology-services-toolkit</artifactId>
<version>1.0.0-SNAPSHOT</version>
<scope>compile</scope>
</dependency>
```


Por lo que debí clonar el repositorio de ontology-services-toolkit y hacer un proyecto de maven también en eclipse de este para que quede en mi repositorio local de Maven (~/.m2/repository), y poder usarlo como dependencia en tu proyecto.

En este repositorio se encuentra un readme donde especifica que para la instalación de este se debe de instalar la biblioteca jrdfox que ya viene en el mismo repositorio. En lib/mvn_install_jrdfox.txt está la información sobre cómo hacerlo. Un error que tuve fue que al poner

```
mvn install:install-file -Dfile=C:\Users\javiw\ontology-services-toolkit\lib\JRDFox.jar -DgroupId=uk.ox.jrdfox -DartifactId=jrdfox -Dversion=1.2776.2017 -Dpackaging=jar
```

no me lo instalaba porque la powershell de windows tomaba los puntos como reparación lo que debí agregar comillas en donde tenía separación con puntos para que lo tomara como algo que venía en conjunto.

Si se descarga una versión de jdk superior a la 8 se deben agregar dependencias en el pom del proyecto ya que estas versiones eliminaron javax.xml.bind. Mi pom editado quedó de la siguiente manera:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>ontology-services-toolkit</groupId>
  <artifactId>ontology-services-toolkit</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <name>ontology-services-toolkit</name>
  <description>ontology services toolkit</description>
  <build>

    <sourceDirectory>src</sourceDirectory>
    <plugins>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.5.1</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
          <encoding>UTF-8</encoding>
        </configuration>
      </plugin>
    </plugins>
  </build>
```

```
<organization>
  <name>University of Oslo</name>
</organization>

<dependencies>
  <!-- Dependencias OWL y otras -->

  <dependency>
    <groupId>net.sourceforge.owlapi</groupId>
    <artifactId>org.semanticweb.hermit</artifactId>
    <version>1.3.8.413</version>
  </dependency>

  <dependency>
    <groupId>net.sourceforge.owlapi</groupId>
    <artifactId>owlapi-distribution</artifactId>
    <version>4.1.3</version>
  </dependency>

  <dependency>
    <groupId>net.sourceforge.owlapi</groupId>
    <artifactId>owlapi-contract</artifactId>
    <version>4.1.3</version>
  </dependency>

  <dependency>
    <groupId>org.semanticweb.elk</groupId>
    <artifactId>elk-owlapi</artifactId>
    <version>0.4.3</version>
  </dependency>

  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.11</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>1.7.18</version>
  </dependency>
```

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>1.7.18</version>
</dependency>

<dependency>
  <groupId>org.eclipse.rdf4j</groupId>
  <artifactId>rdf4j-runtime</artifactId>
  <version>2.2.2</version>
</dependency>

<dependency>
  <groupId>org.json</groupId>
  <artifactId>json</artifactId>
  <version>20171018</version>
</dependency>

<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.0</version>
  <scope>provided</scope>
</dependency>

<!-- Dependencias para JAXB -->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>

<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-core</artifactId>
  <version>2.3.0</version>
</dependency>

<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>2.3.0</version>
</dependency>
```

```

<dependency>
  <groupId>javax.activation</groupId>
  <artifactId>activation</artifactId>
  <version>1.1.1</version>
</dependency>

<!-- JRDFox -->
<dependency>
  <groupId>uk.ox.jrdfox</groupId>
  <artifactId>jrdfox</artifactId>
  <version>1.2776.2017</version>
</dependency>
</dependencies>

<repositories>
  <!-- Repositorio para JRDFox, si es necesario -->
  <repository>
    <id>jrdfox-repo</id>
    <url>http://www.cs.ox.ac.uk/isg/tools/RDFox/</url>
  </repository>
</repositories>
</project>

```

Luego de corregir lo anterior, pude instalar maven para jrdfox así como para ontology-services-toolkit. El último paso fue modificar dentro del script del archivo las rutas de donde obtenía la ontología, así como la ruta donde debían de guardarse los archivos generados. Una vez corrí el archivo se generó lo siguiente



```

1 package ontology_embed;
2
3 import org.semanticweb.owlapi.apibinding.OwlManager;
4
5
6
7
8
9
10
11
12
13 public class ClassAssertion_NormalSplit {
14     public static String onto_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\ontology.owl";
15     public static String train_onto_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\ontology.train.owl";
16     public static String train_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\train.csv";
17     public static String valid_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\valid.csv";
18     public static String test_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\test.csv";
19     public static String class_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\classes.txt";
20     public static String individual_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\individuals.txt";
21     public static String inferred_class_file = "D:\\mail\\cat\\use_cases\\ontology\\data\\inferred_classes.txt";
22
23
24
25
26
27 /**
28  * Those declared (none-inferred) membership axioms are split into train (70%), valid (10%) and test (20%)
29  * A training ontology is created for learning the embedding, by removing valid and test membership axioms
30  * One negative sample is generated for each positive axiom in the training set
31  * @param args
32  */
33 public static void main(String[] args) throws OwlOntologyCreationException, IOException, OwlOntologyStorageException {
34     if(args.length >= 2){
35         onto_file = args[1]; train_onto_file = args[2];
36         train_file = args[3]; valid_file = args[4]; test_file = args[5];
37         class_file = args[6]; individual_file = args[7]; inferred_class_file = args[8];
38     }
39 }
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Problems Servers Terminal Data Source Explorer Properties Console
 <terminated> ClassAssertion_NormalSplit [Java Application] D:\mail\openjdk-8.0.152-h7382act\type\bin\java.exe (19 ene. 2025 11:25:20 - 11:25:31) [pid: 12724]
 Classes: 33, Individuals: 166
 Class assertions #: 213
 train (positive): 149, valid: 21, test: 43

Lo que significa que los archivos fueron generados con éxito.

Cuando ya estaba lista para poder correr ahora si todo y comenzar con el entrenamiento me di cuenta de que mis archivos de prueba, así como de validación tenían una estructura diferente a la que usan ellos (las tripletas no quedaban separadas en cada renglón y cuando empezaban y terminaban se delimitaba esto con <>). Por ello hice un script en python para que eliminara los <> y pusiera una triplete por línea. El script lo deje en la misma carpeta de ontología. El script se ejecuta con el comando:

```
python cleaned.py
```

Después debí modificar en el archivo run_cat_completion el nombre del archivo test y de validación.

Luego de esto tuve unos problemas con los nodos ya que me decía que ciertas dimensiones de mis archivos estaban mal por lo que hice otro script que permite ver si hay inconsistencia entre los nodos

Tuve que eliminar dos nodos que estaban en el archivo test, pero no en el grafo. Leí en el paper de catE que a ellos les había sucedido lo mismo y lo habían resuelto de esa forma por lo que supuse que estaba bien.

En este punto ya tenía todos los archivos que yo creí necesitar, así como las dependencias. Así que corrí el script que ponían en catE para empezar con el entrenamiento.

De todas formas, los últimos días he tenido un problema que no he podido solucionar. Intenté diferentes métodos para debuggear así como también cambiando cosas para ver si se solucionaba, pero no lo conseguí. Mi problema es el siguiente:

```
File "D:\mai\anaconda\envs\cate\lib\site-packages\click\core.py", line 1434,
in invoke

    return ctx.invoke(self.callback, **ctx.params)

File "D:\mai\anaconda\envs\cate\lib\site-packages\click\core.py", line 783,
in invoke

    return __callback(*args, **kwargs)

File "run_cat_completion.py", line 109, in main

    model.train(wandb_logger)

File "D:\mai\catE\use_cases\..\src\cat.py", line 489, in train

    valid_mean_rank, valid_mrr = self.compute_ranking_metrics(mode="validate")
```

```

File "D:\mai\catE\use_cases\..\src\cat_completion.py", line 150, in
compute_ranking_metrics

    print(f"Orderings == Tail: {orderings == tail}")

RuntimeError: The size of tensor a (35) must match the size of tensor b (0) at
non-singleton dimension 0

wandb: 🚀 View run default at: https://wandb.ai/martinezmaite24-facultad-de-intenieria/cate2/runs/b5ys81h1

wandb: Find logs at: wandb\run-20250113_115146-b5ys81h1\logs

```

Lo que sucede es que parece no reconocer la cola en las tuplas generadas, sin embargo, corriendo algunos scripts para verificar si estaban si lo hacían por lo que no entiendo donde pueda estar el problema.

4. Pasos a seguir para conseguir los archivos

En esta sección solo voy a poner el paso a paso lineal para conseguir los archivos, no voy a volver a nombrar las diversas vueltas para llegar a estos, sino que la manera más sencilla.

Antes de comenzar hay que asegurarnos de que nuestra ontología este dentro de la carpeta use_cases dentro de otras subcarpetas con esta estructura ontologia/data/ontologia.owl. Una vez tenemos esto ya podemos comenzar

- i. Crear el archivo edgelist a partir de la ontologia.owl
Dentro de la carpeta catE debemos hacer

```
cd src/
```

Para movernos a la carpeta src

```
python cat_projector.py ruta/a/catE/use_cases/ontologia/data/ontologia.owl
```

Hacemos la proyección en grafo

- ii. Eliminar morfismos innecesarios

Antes de correr el siguiente script se debe entrar al archivo ontologia.edgelist desde algún ide (yo use visual studio) y seleccionar que tenga encoding utf-8 porque sino el siguiente script va a dar error

```
python remove_obvious_morphisms.py
ruta/a/catE/use_cases/ontologia/data/ontologia.edgelist
```

iii. Creación de los archivos necesarios para run_cat_completion

Para esta parte vamos a hacer uso de uno de los archivos de java que usa owl2vec*. Antes de esto es necesario tener creado el proyecto maven (explique más arriba como se hace). Una vez se tiene esto, entrar a eclipse y abrir el archivo ClassAssertion_NormalSplit.java que se encuentra dentro de la carpeta owl2vec*/case_studies/java y poner **run as java application**. Después de esto deberían de aparecer los archivos en la carpeta.

Nota: una cosa que no agregue anteriormente pero abriendo el archivo ClassAssertion_NormalSplit puede verse que cambie las rutas de donde se toman los archivos así como a donde deben dirigirse del archivo original, en mi caso el comiendo dice lo siguiente

```
public class ClassAssertion_NormalSplit {
    public static String onto_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\ontologia.owl";
    public static String train_onto_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\ontologia.train.owl";
    public static String train_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\train.csv";
    public static String valid_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\valid.csv";
    public static String test_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\test.csv";
    public static String class_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\classes.txt";
    public static String individual_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\individuals.txt";
    public static String inferred_class_file =
"D:\\mai\\catE\\use_cases\\ontologia\\data\\inferred_classes.txt";
... resto del archivo
```

Cada uno debe modificar estas rutas por las correspondientes en su computadora.

iv. Correr el archivo run_cat_completion.py

Antes de correr el archivo debemos hacer algunas modificaciones en el mismo así como en el archivo cat.py

En el archivo cat.py hay que modificar desde la línea 135 en la parte de @property agregando el grafo creado para la ontología para cuando se tenga como caso de uso a esta. En mi caso quedo de la siguiente manera

```

@property

def graph_path(self):
    if self._graph_path is not None:
        return self._graph_path
    if "foodon" in self.use_case:
        graph_name = f"{self.use_case}-merged.train.cat_filtered.edgelist"
    elif "go" in self.use_case:
        graph_name = f"{self.use_case}.train.cat_filtered.edgelist"
    elif "ore1" in self.use_case:
        graph_name = f"ORE1.cat.edgelist"
    elif "ontologia" in self.use_case: #desde acá agregué
        graph_name = f"ontologia.cat.edgelist" #hasta acá
    else:
        raise ValueError(f"Unknown use case {self.use_case}")

    if self.el:
        graph_name = graph_name.replace("train", "train_normalized")

    graph_path = os.path.join(self.root, graph_name)
    assert os.path.exists(graph_path), f"Graph file {graph_path} does not exist"
    self._graph_path = graph_path
    return self._graph_path

```

Luego en el archivo run_cat_completion.py se debe agregar ontología como uno de los casos de uso a usar

```

@ck.option('--use_case', '-case', required=True, type=ck.Choice(["go", "foodon", "ontologia"]))

```

Una vez tengamos estas cosas seteadas podemos correr el comando en la powershell de anaconda (teniendo cate activado y estando ubicados en la ruta a los use_cases dentro de la carpeta catE)

```

python run_cat_completion.py --batch_size=8192 --emb_dim=200 --loss_type=normal --lr=0.0001 --margin=1 --num_negs=2 --use_case=ontologia -ns

```


5. Resumen

A lo largo de esta pasantía, di muchos pasos importantes en el estudio e implementación del uso de ontologías en combinación con herramientas de la Web Semántica y técnicas de Machine Learning. Me enfoqué en trabajar con una ontología personalizada aplicada a la herramienta CatE, apoyándome en bibliotecas como mOWL y OWL2Vec* para generar embeddings y estructurar los datos de forma que fueran útiles para los modelos.

Si bien no logré completar todo lo que tenía planeado, como entrenar la red neuronal y generar algunos de los archivos necesarios, avancé muchísimo en el proceso. Este trabajo me permitió entender mejor cómo funcionan estas herramientas, resolver múltiples problemas técnicos, y dejar un camino bastante claro para seguir en el futuro. Sin duda, aún queda mucho por hacer, pero estoy convencida de que este avance marca un buen punto de partida para los próximos pasos.

6. Adjunto emails intercambiados

19/1/25, 20:19

Gmail - Answer about catE

 **Maite Martinez** <martinezmaite24@gmail.com>

Answer about catE
4 messages

Maite Martinez <martinezmaite24@gmail.com> 18 de diciembre de 2024, 14:30
Para: robert.hoehndorf@kaust.edu.sa

Dear Robert, I hope this email finds you well.
I've been recently reading and trying to understand your paper as well as your repository of catE for an internship at my university.
I have some questions about it and hope this does not bother you. The answers are more about the files that are needed to run the catE. I didn't catch it well how I should make the testing files and I was wondering if you could explain more about it.
My best regards, hope you have a happy holidays.
Maite

Robert Hoehndorf <robert.hoehndorf@kaust.edu.sa> 19 de diciembre de 2024, 2:44
Para: Maite Martinez <martinezmaite24@gmail.com>
Cc: Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa>

Dear Fernando,

Could you help Maite, please, with her questions about CatE?

Best,
Rob.
[El texto citado está oculto]

This message and its contents, including attachments are intended solely for the original recipient. If you are not the intended recipient or have received this message in error, please notify me immediately and delete this message from your computer system. Any unauthorized use or distribution is prohibited. Please consider the environment before printing this email.

 1K

Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa> 19 de diciembre de 2024, 6:44
Para: Maite Martinez <martinezmaite24@gmail.com>
Cc: Robert Hoehndorf <robert.hoehndorf@kaust.edu.sa>

Dear Maite,

Thanks for your email and your interest in our work CatE. I am happy to help with your questions. However, I wonder if you are asking about how to reproduce CatE results or if you intend to use CatE on your own datasets.
If you want to reproduce CatE results, you can find the data at <https://zenodo.org/records/13766937> and follow the instructions in the README file in the repository.
If you aim to create your own dataset, you can follow the script https://github.com/bio-ontology-research-group/catE/blob/main/src/create_subsumption_dataset.py, which I just added to the repository since it was missing.
Please let me know if this answers your question or if you have any questions feel free to ask.

Best,
Fernando

On Thu, Dec 19, 2024 at 8:44 AM Robert Hoehndorf <robert.hoehndorf@kaust.edu.sa> wrote:
Dear Fernando,

19/1/25, 20:19

Gmail - Another question about catE

 **Maite Martinez** <martinezmaite24@gmail.com>

Another question about catE
5 messages

Maite Martinez <martinezmaite24@gmail.com> 4 de enero de 2025, 12:34
Para: Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa>

Dear Fernando,

I hope this email finds you well and that you had a wonderful Christmas and a happy New Year!

I have a question regarding which script I could use to generate the necessary files to run CatE. I am trying to use `run_cat_completion.py`, but I couldn't figure out how to create the `valid_cleaned.csv` and `test_cleaned.csv` files. I initially thought they would be generated with the `create_subsumption_dataset.py` script, but it seems that is not the case.

If you could guide me on how to generate these files, it would be greatly appreciated.

Thank you in advance for your help!

Best regards,
Maite

Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa> 5 de enero de 2025, 3:23
Para: Maite Martinez <martinezmaite24@gmail.com>

Dear Maite,

I hope you also had great holidays. Thanks for your question. Certainly, there is a missing script in the repository. I have added it as https://github.com/bio-ontology-research-group/catE/blob/main/src/axioms_to_csv.py. You can use this script to transform the validation and testing owl files to csv.
Let me know if this works for you.

Best,
Fernando
[El texto citado está oculto]

This message and its contents, including attachments are intended solely for the original recipient. If you are not the intended recipient or have received this message in error, please notify me immediately and delete this message from your computer system. Any unauthorized use or distribution is prohibited. Please consider the environment before printing this email.

Maite Martinez <martinezmaite24@gmail.com> 5 de enero de 2025, 7:11
Para: Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa>

Dear Fernando,

Thank you for your quick response and for adding the script to the repository. I appreciate your guidance.

I have a follow-up question regarding the testing and validation setup. When creating the testing and validation datasets, do you typically remove certain axioms from the ontology to test if the model can predict them? Or is there a specific method you recommend for generating these datasets?

This is my first time using CatE, and I want to ensure I am following the best practices.

Thank you for your help!

Best regards, Maite

[El texto citado está oculto]

<https://mail.google.com/mail/u/0/?ik=004fcac47&view=pt&search=all&permthid=thread-a7-2044189627827602146&siml=msg-a7-16294161900...> 1/2

19/1/25, 20:19

Gmail - Answer about catE

Could you help Maite, please, with her questions about CatE?

Best,
Rob.
> Dear Robert, I hope this email finds you well.
> I've been recently reading and trying to understand your paper as
> well as your repository of catE for an internship at my university.
> I have some questions about it and hope this does not bother you. The
> answers are more about the files that are needed to run the catE. I
> didn't catch it well how I should make the testing files and I was
> wondering if you could explain more about it.
> My best regards, hope you have a happy holidays.
> Maite

[El texto citado está oculto]

Maite Martinez <martinezmaite24@gmail.com> 19 de diciembre de 2024, 11:54
Para: Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa>

Dear Fernando,

Thank you for your predisposition and help.
Answering your question, I would like to use catE for my datasets in order to study its results on it. I didn't understand well when I was reading the repository how to make the test files but I will read the file you uploaded and try it.
Thank you again for emailing me back.

Best,
Maite
Maite
[El texto citado está oculto]

19/1/25, 20:19

Gmail - Another question about catE

Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa> 5 de enero de 2025, 8:23
Para: Maite Martinez <martinezmaite24@gmail.com>

Dear Maite,

that is a good question. I followed the procedure in <https://arxiv.org/abs/2009.14654> to generate the datasets. There are other ways to test the methods, for example testing on the deductive closure of the training set. Testing in the deductive closure will check how well the method can do reasoning tasks, while testing in missing axioms (the ones removed) will test if the method can leverage patterns in the training set (similar as in link prediction in knowledge graphs). So I am not sure what is the "best practice" here because it also depends on your use cases and datasets. I hope this answer can be of some help.

Best,
Fernando
[El texto citado está oculto]

Maite Martinez <martinezmaite24@gmail.com> 5 de enero de 2025, 17:53
Para: Fernando Zhapacamacho <fernando.zhapacamacho@kaust.edu.sa>

Thanks for your help. I will read it and try to do it.

Best, Maite
[El texto citado está oculto]